

08.3: Apache Hadoop



UMD DATA605 - Big Data Systems

08.3: Apache Hadoop

- **Instructor:** Dr. GP Saggese -
- @References@
 - Ghemawat et al.: *The Google File System*, 2003
 - Dean et al.: *MapReduce: Simplified Data Processing on Large Clusters*, 2004

2 / 8: Hadoop Ecosystem (aka Hadoop Zoo)

Hadoop Ecosystem (aka Hadoop Zoo)

- **Hadoop MapReduce**
 - A framework for processing large data sets in parallel across a Hadoop cluster
- **HDFS**
 - Distributed file system
- **Pig**
 - High-level data-flow framework for parallel computation
- **HBase**
 - Scalable, distributed database
 - Structured data storage for large tables (like Google BigTable)
- **Cassandra**
 - Scalable multi-master database with no single points of failure
- **Hive**
 - Data warehouse infrastructure
 - Provide data summarization and ad-hoc querying
- **ZooKeeper**



2 / 8

- **Hadoop MapReduce**
 - *Framework for processing large data sets:* MapReduce allows for the parallel processing of vast amounts of data across a Hadoop cluster. It breaks down tasks into smaller sub-tasks, processes them simultaneously, and then combines the results.
- **HDFS (Hadoop Distributed File System)**
 - *Distributed file system:* HDFS is designed to store large data sets reliably and to stream those data sets at high bandwidth to user applications. It splits files into blocks and distributes them across the cluster.
- **Pig**
 - *High-level data-flow framework:* Pig simplifies the writing of complex data transformations using a high-level language called Pig Latin. It is particularly useful for processing and analyzing large data sets in parallel.
- **HBase**
 - *Scalable, distributed database:* HBase is a NoSQL database that provides real-time read/write access to large datasets. It is modeled after Google's BigTable and is suitable for sparse data sets.
- **Cassandra**
 - *Scalable multi-master database:* Cassandra is designed to handle large amounts of data across many commodity servers, providing high availability with no single point of failure.
- **Hive**
 - *Data warehouse infrastructure:* Hive facilitates querying and managing large datasets residing in distributed storage using SQL-like syntax. It is ideal for data summarization and ad-hoc querying.
- **ZooKeeper**

- *Coordination service for distributed applications:* ZooKeeper provides a centralized service for maintaining configuration information, naming, providing distributed synchronization, and group services.
- **YARN, Kafka, Storm, Spark, Solr, ...**
 - These are additional components in the Hadoop ecosystem that provide various functionalities such as resource management (YARN), real-time data streaming (Kafka, Storm), in-memory processing (Spark), and search capabilities (Solr). Each tool serves a specific purpose, enhancing the overall capabilities of the Hadoop ecosystem.

3 / 8: Hadoop Distributed File System (HDFS)

Hadoop Distributed File System (HDFS)

- HDFS is a **distributed file system**
 - Designed to store large data sets reliably
 - Part of the Apache Hadoop ecosystem
 - Inspired by the Google File System (GFS)
- 1. Optimized for **high-throughput access** to large files
 - Suitable for batch processing
 - Not low-latency access
- 2. Designed for **fault tolerance and scalability**
 - Ensures fault tolerance through replication
 - Blocks are stored on different nodes and racks
 - Provides data availability even if some nodes fail
 - Follows a primary-secondary architecture
 - Replication strategy improves read performance

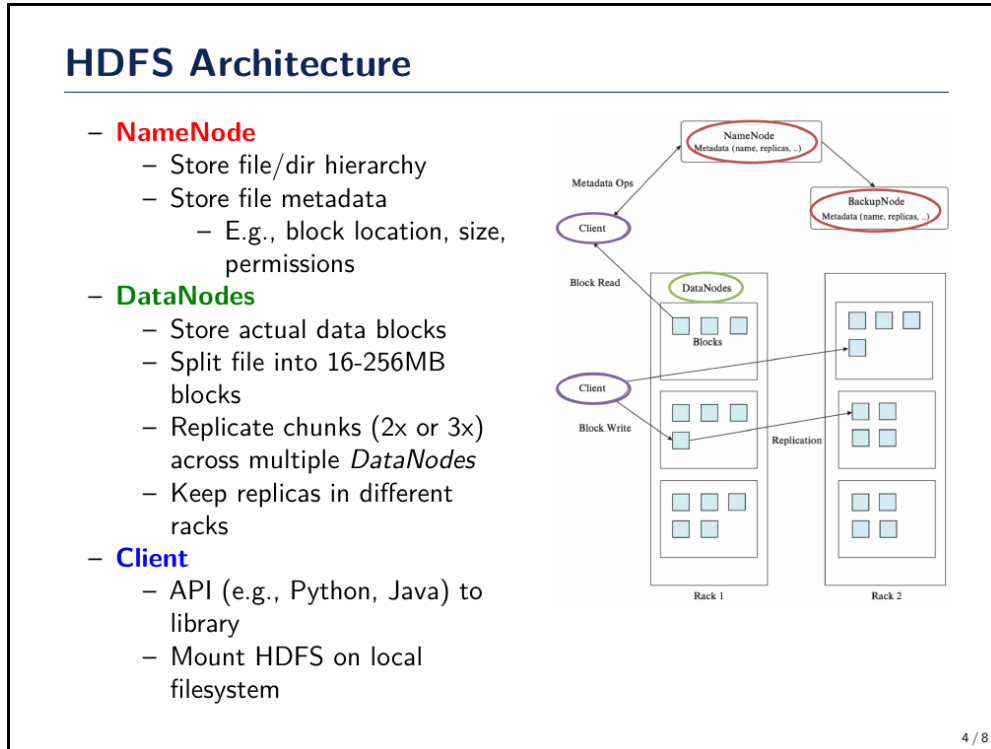


3 / 8

- **HDFS is a *distributed file system***
 - It is designed to store large data sets reliably, making it a crucial part of the Apache Hadoop ecosystem.
 - HDFS is inspired by the Google File System (GFS), which was one of the first systems to address the challenges of storing and processing large amounts of data across many machines.
- 1. **Optimized for *high-throughput access to large files***
 - HDFS is particularly suitable for batch processing, where large volumes of data are processed in chunks.
 - It is not designed for low-latency access, meaning it is not ideal for applications that require quick data retrieval.
- 2. **Designed for *fault tolerance and scalability***
 - HDFS ensures fault tolerance through a replication strategy, where data blocks are stored on multiple nodes and racks. This ensures data availability even if some nodes fail.
 - It follows a primary-secondary architecture, which helps in managing data efficiently.

- The replication strategy not only provides fault tolerance but also improves read performance, as data can be read from multiple sources.

4 / 8: HDFS Architecture



- **NameNode**
 - Acts as the master server in HDFS, managing the file system namespace.
 - Stores metadata about files and directories, such as their hierarchy and attributes.
 - *Metadata* includes details like block locations, file sizes, and permissions.
- **DataNodes**
 - Responsible for storing the actual data blocks of files.
 - Files are divided into blocks ranging from 16MB to 256MB.
 - Each block is replicated, typically 2 or 3 times, across different DataNodes to ensure reliability.
 - Replicas are distributed across different racks to enhance fault tolerance.
- **Client**
 - Provides an interface for users to interact with HDFS, often through APIs in languages like Python or Java.
 - Allows HDFS to be mounted on a local filesystem, making it accessible as if it were part of the local storage.

The diagram illustrates the interaction between these components. The NameNode handles meta-data operations, while DataNodes manage block storage and replication. Clients perform read and write operations, interacting with both NameNode and DataNodes. Replication across racks ensures data durability and availability.

5 / 8: HDFS: Read / Write Protocols

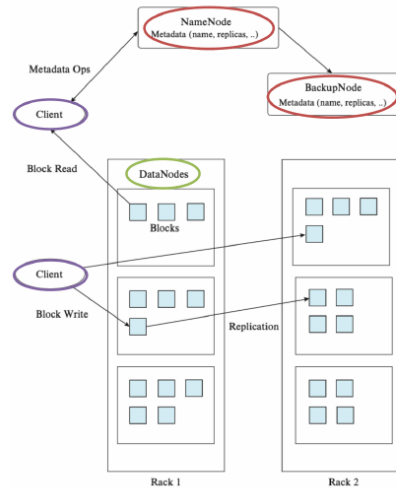
HDFS: Read / Write Protocols

– Read

- Contact *NameNode* for *DataNode* and block pointer
- Choose nearest *DataNode* for each block
- Connect to *DataNode* for data access
- Read blocks in parallel to improve performance
- Client reassembles data in correct order

– Write

- *NameNode* creates blocks
- Assign blocks to multiple *DataNodes*
- Client sends data to *DataNodes*
- *DataNodes* store data
- Pipeline blocks to other replicas



5 / 8

– Read Protocol

- The client first contacts the *NameNode* to get information about which *DataNode* holds the data and the specific block pointers. This is crucial because the *NameNode* manages metadata and knows where each piece of data is stored.
- The client selects the nearest *DataNode* for each block to minimize latency and improve data retrieval speed.
- A connection is established with the *DataNode* to access the data directly.
- Blocks are read in parallel, which significantly boosts performance by allowing simultaneous data retrieval.
- The client then reassembles the blocks in the correct order to reconstruct the original data file.

– Write Protocol

- The *NameNode* is responsible for creating new blocks and assigning them to various *DataNodes*. This ensures data is distributed and managed efficiently.
- The client sends the data to the assigned *DataNodes*, where it is stored.
- The *DataNodes* then replicate the data to other nodes, ensuring redundancy and fault tolerance.
- The write operation is considered successful only after all replicas acknowledge receipt of the data, ensuring data integrity and reliability.

6 / 8: Fault Tolerance and Recovery

Fault Tolerance and Recovery

- *NameNode* monitors *DataNode* heartbeat signals
 - On failure, blocks are re-replicated to maintain replication factor
- *NameNode* itself is a single point of failure
 - Solved with HDFS High Availability
- Data integrity ensured using checksums

6 / 8

- **Fault Tolerance and Recovery**
- *NameNode* monitors *DataNode* heartbeat signals
 - In a distributed file system like HDFS (Hadoop Distributed File System), the *NameNode* is responsible for managing the metadata and directory structure of the file system. It keeps track of where data is stored across the cluster.
 - *DataNodes* are the nodes where actual data is stored. They send regular heartbeat signals to the *NameNode* to indicate they are functioning properly.
 - If a *DataNode* fails or stops sending heartbeat signals, the *NameNode* detects this failure. To ensure data is not lost, the system automatically re-replicates the data blocks that were stored on the failed *DataNode* to other healthy nodes. This process helps maintain the desired replication factor, which is crucial for data reliability and availability.
- *NameNode* itself is a single point of failure
 - The *NameNode* is critical because it holds the metadata of the entire file system. If it fails, the entire system can become inaccessible.
 - To address this vulnerability, HDFS High Availability (HA) is implemented. HA involves having a standby *NameNode* that can take over if the active *NameNode* fails, thus eliminating the single point of failure and ensuring continuous operation of the system.
- **Data integrity ensured using checksums**
 - Data integrity is crucial in any storage system to ensure that the data read is the same as the data written.
 - HDFS uses checksums to verify data integrity. When data is written to the system, a

checksum is calculated and stored. Later, when data is read, the checksum is recalculated and compared to the stored checksum to ensure the data has not been corrupted or altered. This mechanism helps in detecting and correcting errors, ensuring reliable data storage.

7 / 8: HDFS vs Traditional File Systems

HDFS vs Traditional File Systems

- Best for **storing and processing large-scale files**
 - E.g., logs, media, sensor data
 - Commonly used in data lakes and ETL pipelines
 - Supports very large files and directories
 - Performance degrades with many small files
- Optimized for **write-once, read-many** access pattern
- Lacks low-latency access, but provides **high throughput**
 - Good for analytics (OLAP)
 - Not suitable for transactional systems (OLTP)
 - E.g., bank

7 / 8

- **HDFS vs Traditional File Systems**
- Best for *storing and processing large-scale files*
 - HDFS, or Hadoop Distributed File System, is designed to handle very large files, such as logs, media files, and sensor data. This makes it ideal for environments where you need to store and process massive amounts of data, like in data lakes or during ETL (Extract, Transform, Load) processes.
 - It can efficiently manage large files and directories, but it struggles with performance when dealing with a large number of small files. This is because HDFS is optimized for handling big chunks of data rather than numerous tiny ones.
- Optimized for *write-once, read-many* access pattern
 - HDFS is particularly suited for scenarios where data is written once and read multiple times. This is common in data analysis tasks where data is collected, stored, and then repeatedly accessed for insights.
- Lacks low-latency access, but provides *high throughput*
 - While HDFS offers high throughput, making it great for analytical processing (OLAP - Online Analytical Processing), it is not designed for low-latency access. This means it's

not suitable for systems that require quick, real-time data access, such as transactional systems (OLTP - Online Transaction Processing) like those used in banking.

8 / 8: MapReduce: Hadoop

MapReduce: Hadoop

- **Hadoop**: open-source MapReduce implementation



- **Functionalities**
 - Partition input data (HDFS)
 - Input adapters
 - E.g., HBase, MongoDB, Cassandra, Amazon Dynamo
 - Schedule program execution across machines
 - Handle machine failures
 - Manage inter-machine communication
 - Perform *GroupByKey* step
 - Output adapters
 - E.g., Avro, ORC, Parquet
 - Schedule multiple *MapReduce* jobs

8 / 8

- **Hadoop**: This is an open-source implementation of the MapReduce programming model. It allows for the processing of large data sets across clusters of computers using simple programming models. Hadoop is designed to scale up from a single server to thousands of machines, each offering local computation and storage.
- **Functionalities**:
 - **Partition input data (HDFS)**: Hadoop uses the Hadoop Distributed File System (HDFS) to split large data sets into smaller, manageable pieces that can be processed in parallel.
 - **Input adapters**: These allow Hadoop to read data from various sources like HBase, MongoDB, Cassandra, and Amazon Dynamo, making it versatile in handling different data formats.
 - **Schedule program execution across machines**: Hadoop efficiently distributes tasks across multiple machines, ensuring that the workload is balanced and resources are utilized effectively.
 - **Handle machine failures**: Hadoop is designed to be fault-tolerant, automatically managing failures by redistributing tasks from failed nodes to healthy ones.
 - **Manage inter-machine communication**: It ensures that data is correctly transferred between machines during processing, which is crucial for the *MapReduce* operations.
 - **Perform *GroupByKey* step**: This is a key operation in MapReduce where data is grouped by key, allowing for aggregation and analysis.

- **Output adapters:** These enable Hadoop to write processed data into various formats like Avro, ORC, and Parquet, which are optimized for different use cases.
- **Schedule multiple *MapReduce* jobs:** Hadoop can manage and execute multiple jobs in sequence or parallel, optimizing the processing of large data sets.